

TUGAS KPL

Nama : Chiara Calina Devi

Kelas : SE08-01

NIM : 103122400016

Github :

https://github.com/chiaracn/KPL_CHIARACALINADEVI_103122400016_SE0801/tree/main/10_Library_Construction/soal

Kode : main.py

```
import time

data_mahasiswa = [
    {"nama": "Budi", "tugas": 80, "uts": 75, "uas": 90},
    {"nama": "Ani", "tugas": 60, "uts": 65, "uas": 70},
    {"nama": "Citra", "tugas": 90, "uts": 85, "uas": 95},
    {"nama": "Doni", "tugas": 50, "uts": 55, "uas": 60},
    {"nama": "Eka", "tugas": 75, "uts": 80, "uas": 85},
]

def validasi_nilai(nilai):
    if nilai is None:
        return False
    if not isinstance(nilai, (int, float)):
        return False
    if nilai < 0 or nilai > 100:
        return False
    return True

def hitung_nilai_akhir(mahasiswa):
    if "tugas" not in mahasiswa or "uts" not in mahasiswa or "uas" not in mahasiswa:
        raise ValueError(f'Data mahasiswa {mahasiswa.get("nama", "Tanpa Nama")} tidak lengkap.')

    tugas = mahasiswa["tugas"]
    uts = mahasiswa["uts"]
    uas = mahasiswa["uas"]

    if not (validasi_nilai(tugas) and validasi_nilai(uts) and validasi_nilai(uas)):
        raise ValueError(f'Nilai {mahasiswa.get("nama", "Tanpa Nama")} tidak valid.')

    nilai_akhir = (tugas * 0.3) + (uts * 0.3) + (uas * 0.4)
    return nilai_akhir

def cek_kelulusan(nilai_akhir):
    return "Lulus" if nilai_akhir >= 70 else "Tidak Lulus"

def hitung_rata_rata_kelas(data):
    total = 0
    jumlah_mahasiswa = 0

    for mahasiswa in data:
        nilai_akhir = hitung_nilai_akhir(mahasiswa)
```

```

        total += nilai_akhir
        jumlah_mahasiswa += 1

    return total / jumlah_mahasiswa if jumlah_mahasiswa > 0 else 0

def cari_mahasiswa_lulus(data):
    mahasiswa_lulus = []
    for mahasiswa in data:
        nilai_akhir = hitung_nilai_akhir(mahasiswa)
        if cek_kelulusan(nilai_akhir) == "Lulus":
            mahasiswa_lulus.append(mahasiswa["nama"])
    return mahasiswa_lulus

def main():
    start = time.perf_counter()

    for mahasiswa in data_mahasiswa:
        try:
            nilai_akhir = hitung_nilai_akhir(mahasiswa)
            status = cek_kelulusan(nilai_akhir)
            print(f'{mahasiswa["nama"]} - Nilai Akhir: {nilai_akhir:.2f} - Status: {status}')
        except ValueError as e:
            print(e)

    print(f'Rata-rata kelas: {hitung_rata_rata_kelas(data_mahasiswa):.2f}')
    print("Mahasiswa lulus:", cari_mahasiswa_lulus(data_mahasiswa))

    end = time.perf_counter()
    print(f"Waktu eksekusi: {end - start:.6f} detik")

if __name__ == "__main__":
    main()

```

Test_main.py

```

import unittest
from main import hitung_nilai_akhir, cek_kelulusan, validasi_nilai

class TestNilaiMahasiswa(unittest.TestCase):
    def test_hitung_nilai_akhir_budi(self):
        data = {"nama": "Budi", "tugas": 80, "uts": 75, "uas": 90}
        self.assertAlmostEqual(hitung_nilai_akhir(data), 82.5)

    def test_hitung_nilai_akhir_ani(self):
        data = {"nama": "Ani", "tugas": 60, "uts": 65, "uas": 70}
        self.assertAlmostEqual(hitung_nilai_akhir(data), 65.5)

    def test_cek_kelulusan_lulus(self):
        self.assertEqual(cek_kelulusan(70), "Lulus")

    def test_cek_kelulusan_tidak_lulus(self):
        self.assertEqual(cek_kelulusan(69.99), "Tidak Lulus")

```

```
def test_validasi_nilai_none(self):
    self.assertFalse(validasi_nilai(None))

if __name__ == "__main__":
    unittest.main()
```

Output :

```
E:\semester 4\KPL\KPL_CHIARACALINADEVI_103122400016_SE0801\10_Library_Construction\soal>python main.py
Budi - Nilai Akhir: 82.50 - Status: Lulus
Ani - Nilai Akhir: 65.50 - Status: Tidak Lulus
Citra - Nilai Akhir: 90.50 - Status: Lulus
Doni - Nilai Akhir: 55.50 - Status: Tidak Lulus
Eka - Nilai Akhir: 80.50 - Status: Lulus
Rata-rata kelas: 74.90
Mahasiswa lulus: ['Budi', 'Citra', 'Eka']
Waktu eksekusi: 0.000988 detik

E:\semester 4\KPL\KPL_CHIARACALINADEVI_103122400016_SE0801\10_Library_Construction\soal>python -m unittest -v test_main.py
test cek kelulusan_lulus (test_main.TestNilaiMahasiswa.test_cek_kelulusan_lulus) ... ok
test cek kelulusan_tidak_lulus (test_main.TestNilaiMahasiswa.test_cek_kelulusan_tidak_lulus) ... ok
test hitung_nilai_akhir_anis (test_main.TestNilaiMahasiswa.test_hitung_nilai_akhir_anis) ... ok
test hitung_nilai_akhir_budi (test_main.TestNilaiMahasiswa.test_hitung_nilai_akhir_budi) ... ok
test_validasi_nilai_none (test_main.TestNilaiMahasiswa.test_validasi_nilai_none) ... ok

-----
Ran 5 tests in 0.002s

OK

E:\semester 4\KPL\KPL_CHIARACALINADEVI_103122400016_SE0801\10_Library_Construction\soal>
```

1. Performance Analysis

Program ini berjalan cukup cepat karena hanya memproses 5 data mahasiswa. Waktu eksekusi diukur menggunakan fungsi time, yaitu dengan mencatat waktu sebelum dan sesudah program dijalankan. Hasilnya menunjukkan bahwa program dieksekusi dalam waktu yang sangat singkat (hanya beberapa milidetik), sehingga performanya sudah cukup baik untuk jumlah data kecil.

2. Code Tuning

Nilai akhir sebaiknya disimpan ke dalam variabel agar tidak dihitung berulang-ulang. Soalnya, nilai akhir dipakai di beberapa bagian seperti untuk menentukan kelulusan, menghitung rata-rata, dan mencari mahasiswa yang lulus. Kalau dihitung terus-menerus, program jadi tidak efisien. Dengan menyimpannya sekali saja, program jadi lebih cepat dan kode juga lebih rapi.

3. Unit Testing

Dilakukan 5 pengujian untuk memastikan program berjalan dengan benar, yaitu:

- Menguji perhitungan nilai akhir mahasiswa
- Menguji hasil nilai akhir untuk data lain
- Menguji kondisi mahasiswa yang lulus
- Menguji kondisi mahasiswa yang tidak lulus
- Menguji validasi nilai yang tidak valid (seperti None)

Hasilnya, semua pengujian berhasil tanpa error, jadi fungsi-fungsi dalam program sudah bekerja dengan baik.

4. Debugging

Jika nilai uas bernilai None, program akan error karena None bukan angka dan tidak bisa digunakan dalam operasi matematika. Saat program mencoba menghitung `None * 0.4`, Python akan menampilkan error (TypeError). Oleh karena itu, perlu dilakukan pengecekan terlebih dahulu sebelum perhitungan dilakukan.

5. Perbaikan

Agar program tidak error, perlu ditambahkan validasi pada setiap nilai. Validasi ini memastikan bahwa:

- Nilai tidak boleh None
- Nilai harus berupa angka
- Nilai harus berada di rentang 0 sampai 100

Jika ada data yang tidak valid, program sebaiknya menampilkan pesan error yang jelas, bukan langsung berhenti. Dengan begitu, program jadi lebih aman dan tidak mudah crash.